2771N

Continuous Pursuit-Evasion with Aerodynamic Constraints

Computer Science Tripos – Part II

April 24, 2026

Declaration of Originality

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number **2771N**, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. I am content for my report to be made available to the students and staff of the University.

Date: April 24, 2026

Proforma

Candidate number:	2771N
Project Title:	**Continuous Pursuit-Evasion with Aerodynamic Constraints**
Examination:	Computer Science Tripos – Part II, 2026
Word Count:	WORD COUNT[1]
Code Line Count:	LINE COUNT[2]
Project Originator:	The candidate
Day-To-Day Supervisor:	Ramsey Faragher
Marking Supervisor:	Alastair Beresford

Original Aims of the Project

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Fusce ac turpis quis ligula lacinia aliquet. Mauris ipsum. Nulla metus metus, ullamcorper vel, tincidunt sed, euismod in, nibh.

Work Completed

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Fusce ac turpis quis ligula lacinia aliquet. Mauris ipsum. Nulla metus metus, ullamcorper vel, tincidunt sed, euismod in, nibh.

Special Difficulties

None.

[1] This word count was computed using `texcount`.
[2] This code line count was computed using `tokei` (excluding autogenerated test output).

Contents

1 Introduction **1**

2 Preparation **2**

3 Implementation **3**

3.1 Repository Overview 3

3.2 Kinematic Model 4

3.2.1 One- and Two-Dimensionality 5

3.2.2 Three-Dimensionality 6

3.2.3 Capturing 7

3.3 Decision Making 8

3.3.1 FastMDP 8

3.3.2 Parameter Tuning 8

3.3.3 Reward Functions 8

3.4 Output System 9

3.4.1 Terminal Dashboard 9

3.5 Extensions 10

4 Evaluation **12**

5 Conclusions **13**

Bibliography **14**

A ⟨Sprint Reviews⟩ **15**

B ⟨Configuration Values⟩ **16**

C ⟨Validation Scenarios⟩ **17**

D Proposal **18**

Chapter 1

Introduction

Chapter 2

Preparation

Chapter 3

Implementation

3.1 Repository Overview

Path	Description
`src/main.py`	Entry point for individual simulation runs.
`src/tune.py`	Entry point for parameter-tuning simulation runs.
`src/validate.py`	Entry point for behavioural and kinematic validation simulation runs.
`src/simulation/simulation.py`	Simulation kinematics and lifecycle manager.
`src/simulation/mdp.py`	MDP framework, FastMDP implementation and reward functions.
`src/outputs/base.py`	Output manager and base output type component.
`src/outputs/*.py`	Output type implementations (e.g. plot, video).
`src/display.py`	Terminal dashboard set-up and lifecycle.
`src/visualisation.py`	`pyvista` interactive 3D visualisation.
`results/`	Directory for any produced outputs.
`src/assets/`	Public-use models for use in 3D visualisation.
`src/configs/`	Configuration values for project components.
`tests/`	Unit tests for project components.

Table 3.1: Repository structure

The simulation and machine learning functionality were coded from scratch without reliance on pre-existing code or external libraries. A selection of features from the Python standard library and mainstream libraries were used:

- `rich` for clear and colourful terminal logs and interfaces.
- `pytest` for unit test management.
- `numpy` for efficient N-dimensional numerical computation.
- `matplotlib` and `asciichartpy` for standard data plotting functionality.
- `pyvistaqt` and `pyqt5` for the interactive 3D visualisation.
- `numba` for JIT compilation of computationally-heavy Python code to machine code at runtime (described further in Extensions).

In all cases, the relevant library provided generic functionality that assisted with performance or visualisation, and additional code was written to interoperate with the simulation implementation.

The project supported configuration of many key values. Unless otherwise stated, all examples and evaluation in this dissertation use the base configuration included in Appendix C.

Before beginning development, the project scaffolding was set up. A `pytest` configuration was set up with a dedicated test folder importing source components, and a GitHub Actions workflow was configured to run the full test suite automatically on every push to the remote repository. The `argparse` standard library was used in a `main.py` file to provide a base command-line interface for configuring and launching simulation runs.

Figure 3.3 gives an architectural overview of the final system, explored in detail throughout this chapter.

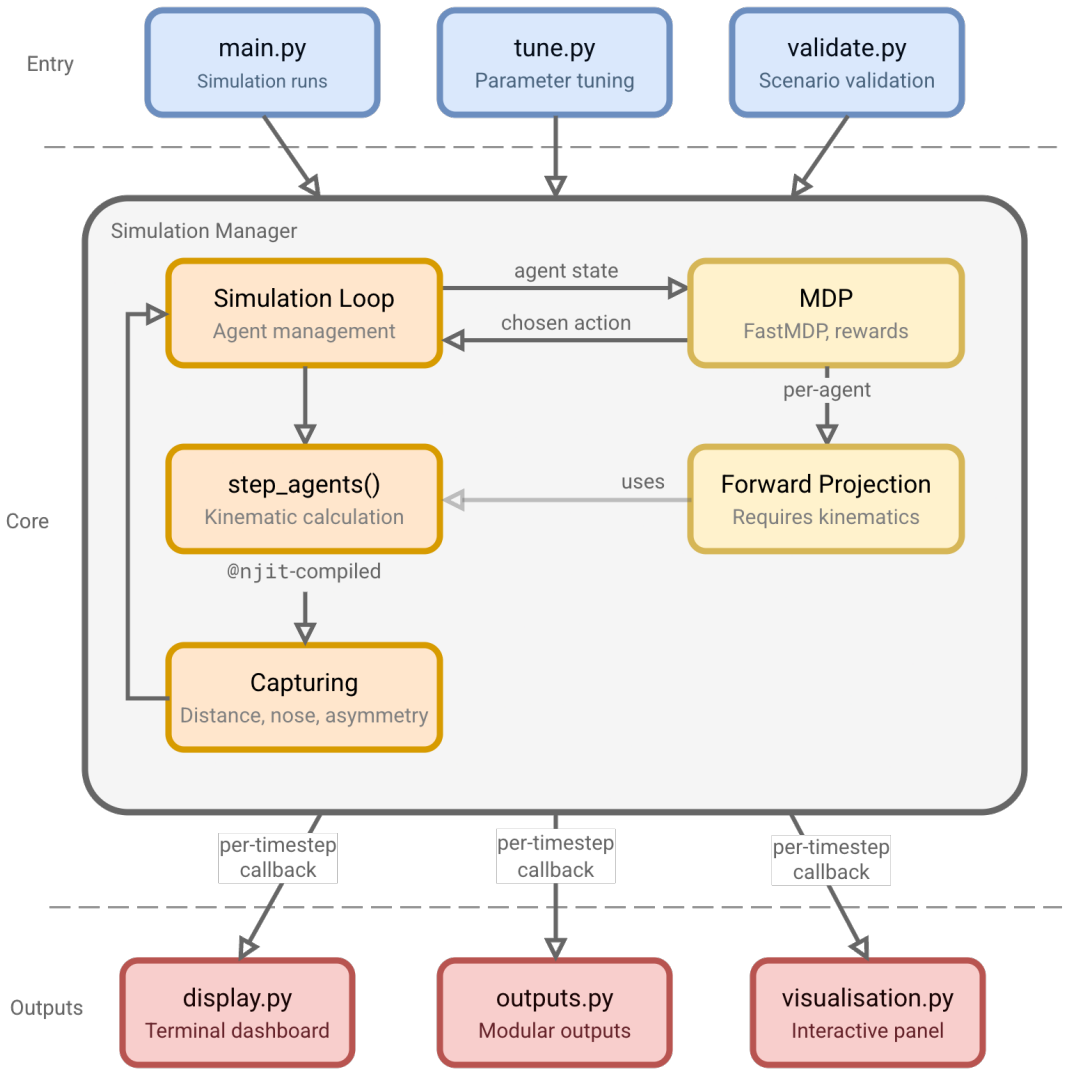

Figure 3.1: System architecture

3.2 Kinematic Model

The kinematic model was developed in incremental steps up to three-dimensionality with pursuit-evasion behaviour expanded at each stage. This allowed for smoother integration of

other components (outputs, visualisations), and for the decision-making functionality to be examined and debugged in much simpler environments before presenting a full solution. The target was a full three-dimensional pseudo-6DoF model as described in the Preparation chapter.

In all cases, time was discretised with a `STEPS_PER_SECOND` configuration value to give a timestep variable $dt = 1/\texttt{STEPS_PER_SECOND}$ for kinematic calculations. Each kinematic value can be updated as $x_{t+1} = x_t + \dot{x} \cdot dt$ through semi-implicit Euler integration for multiple equations, following the discrete-time approach of Bertram et al.[1].

3.2.1 One- and Two-Dimensionality

The aim of one-dimensional kinematics was primarily to validate the simulation timestepping framework and agent system. The space is defined with a right-positive convention and agents are represented with a position and a velocity. The action space is a set of discrete acceleration (m/s^2) values $\{a_1, a_2, \ldots, a_n\}$ with velocity (m/s) updated as $v_{t+1} = v_t + a_{\text{chosen}} \cdot dt$ and position (m) as $p_{t+1} = p_t + v_{t+1} \cdot dt$, all in base SI units. The system interfaced with a naive version of the *FastMDP* algorithm described in section 3.3 and a simple output system, and the resulting trajectories shown in fig. 3.2 confirmed that a decision-making agent approaches its opponent.

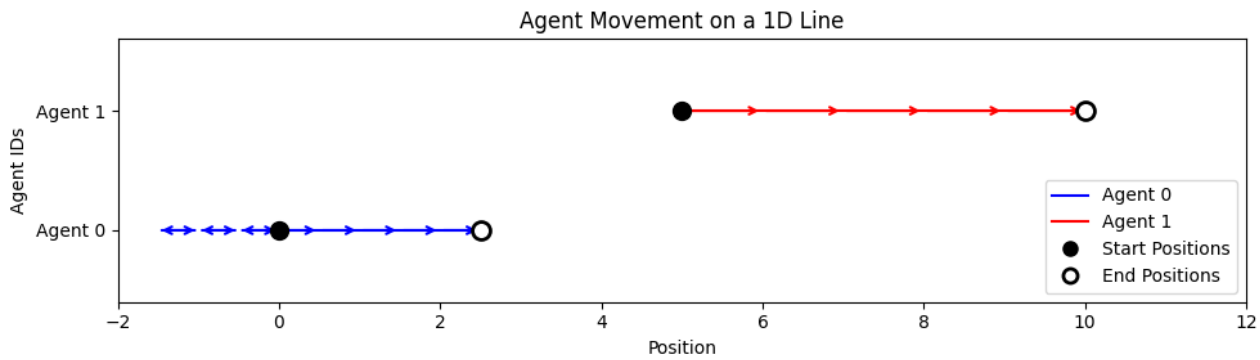

Figure 3.2: 1D Simulation Plot

[REVIEW: do i need to do anymore work on this plot? its just here in its raw state as a development aid, and its effectivness as an visualiastion is considered in Evaluation (spoiler alert: its not great)]

The codebase was initially object-oriented as determined in my design preparation. A `Simulation` singleton maintained a list of `Agent` objects that encapsulated their own kinematic state. This was a natural design given the conceptual clarity of individual agent entities. The decision-making framework also supported this design, as per-agent MDPs could be instantiated from a single class with relevant parameters. A `step()` method could be run each timestep to determine actions, apply them and update kinematic values for each agent.

It became apparent in early development that this design was incompatible with eventual intended optimisations. Individual agent object updates could not take advantage of NumPy's array broadcasting, and Numba's JIT compilation incurred a significant performance penalty when interfacing with arbitrary objects rather than raw procedures.

The aim of two-dimensional kinematics was to more deeply explore the *FastMDP* algorithm and possible reward functions. This would require a base level of optimisation, so I decided to

refactor the codebase design to address the above issues. Kinematic calculations were extracted to a single `step_agents()` method that accepted flat arrays of agent state and applied updates across all N agents simultaneously using NumPy's vectorised operations. This was also a clean interface for later investigating Numba's no-Python JIT compilation mode (described in section 3.4).

Two-dimensional kinematics introduced a heading parameter ψ measured in radians, replaced acceleration with thrust value n_x and expanded position and velocity to 2-vectors. The action space became $\{(\dot{\psi}, n_x)\}$, and the numerical integration is shown in listing 1.

```
# input values: heading_rates, thrusts (applied across N agents)
headings = headings + heading_rates * dt

ax = thrusts * np.cos(headings)
ay = thrusts * np.sin(headings)
accelerations = np.stack((ax, ay), axis=-1)

velocities = velocities + accelerations * dt
positions = positions + velocities * dt

return (positions, velocities, headings)
```

Listing 1: 2D kinematic updates via semi-implicit Euler integration using vectorised operations.

The 2D exploration of pursuit-evasion behaviour is described in section 3.3.

[TODO: video artefact of 2d pursuit evasion to include?]

3.2.2 Three-Dimensionality

Three-dimensional kinematics were introduced midway through the project, after a modular output system and the majority of the MDP framework had been implemented.

The 3D model significantly expanded on the 2D implementation. The agent state grew to include speed V, attack angle α, flight path angle γ, roll angle ϕ, and azimuth angle ψ in addition to position, following the pseudo-6DoF model described in the Preparation chapter. The action space became $\{(n_x, \dot{\alpha}, \dot{\phi})\}$ with thrust n_x, attack angle rate $\dot{\alpha}$ and roll angle rate $\dot{\phi}$ as inputs. $n_f = n_x \sin\alpha + L$ is derived from these, where L is a configured lift constant.

The rate calculations from Bertram and Wei [1] are implemented directly:

```
speed_rates =
    g *
    (thrusts * np.cos(attack_angles) - np.sin(flight_path_angles))

flight_path_angles_rates =
    (g / speeds) *
    (nf * np.cos(roll_angles) - np.cos(flight_path_angles))

azimuth_angles_rates =
    (g * nf * np.sin(roll_angles)) /
    (speeds * np.cos(flight_path_angles))
```

Listing 2: 3D kinematic rate calculations following the pseudo-6DoF model of [1].

Realistic kinematic bounds are enforced through value clipping after integration, with configurable minimums and maximums per-agent to compare parameters. The speed and angular state are resolved to a velocity vector for the position update:

```
speeds = np.clip(speeds + speed_rates * dt, speed_mins, speed_maxs)

vx = speeds * np.cos(flight_path_angles) * np.cos(azimuth_angles)
vy = speeds * np.cos(flight_path_angles) * np.sin(azimuth_angles)
vz = speeds * np.sin(flight_path_angles)
velocities = np.stack((vx, vy, vz), axis=-1)
positions = positions + velocities * dt
```

Listing 3: Scalar speed and angles resolved to a 3D velocity vector for position update.

The `step_agents()` function is fully compatible with the `@njit` decorator for Numba's no-Python mode, discussed further in section 3.4.

[REVIEW: three dimensional output? video? plot? visualisation screenshot?]

3.2.3 Capturing

The capturing mechanic tied into the kinematic system. A simulation ends after a maximum number of timesteps, or when all but one agent has been captured. The capturing condition laid out during preparation involved being within a configurable distance of a control point 3 seconds behind an enemy, and aligned towards them within some limit. I found this did not result in accurate-looking capture during examination of the visualisation, so redesigned the criteria during development.

The following must hold simultaneously true for N consecutive timesteps for a capture to occur:

1. **Distance check**: the Euclidean distance between agents falls within a capture radius r.
2. **Nose check**: the pursuer's nose direction is within 50° of the evader.
3. **Asymmetry check**: the pursuer has a meaningfully better alignment than the evader, $a_{\text{pursuer}} > a_{\text{evader}} + 0.2$.

Fragments of implementation can be seen in listing 4. These conditions also guided the eventual design of the reward functions in section 3.4.

```
r_pe = evader_position - pursuer_position
d = np.linalg.norm(r_pe)
r_pe_hat, r_ep_hat = r_pe / d, -r_pe / d

pursuer_alignment = np.dot(pursuer_nose, r_pe_hat)
evader_alignment = np.dot(evader_nose, r_ep_hat)

distance_check = d < SimulationConfig.CAPTURE_RADIUS
nose_check = pursuer_alignment >= np.cos(np.deg2rad(50))
asymmetry_check = pursuer_alignment > evader_alignment + 0.2
```

Listing 4: Capture condition values and comparisons

Captures are determined at each timestep within the main simulation loop:

```
while self.simulation.timestep < SimulationConfig.MAX_TIMESTEPS:
    self.simulation.step()
    captures = self.simulation.determine_captures()

    for captured_id, capturer_id in captures:
        all_captures.append(
            (self.simulation.timestep, captured_id, capturer_id)
        )

    for callback in callbacks:
        callback(self.simulation)

    if np.sum(self.simulation.active) <= 1:
        break
```

Listing 5: Simulation loop with capture-breaking logic and per-timestep callbacks

3.3 Decision Making

3.3.1 FastMDP

3.3.2 Parameter Tuning

3.3.3 Reward Functions

used python debugger!!

3.4 Output System

Being able to visualise simulation runs in various ways was a key requirement during development. Qualitative analysis of flight paths, trajectories and agent interactions helped me shape rewards and debug kinematic issues.

3.4.1 Terminal Dashboard

A terminal dashboard was created after implementing the core simulation timestep framework and one-dimensional kinematics. The core functionality was a panel to display key values of the. In addition two

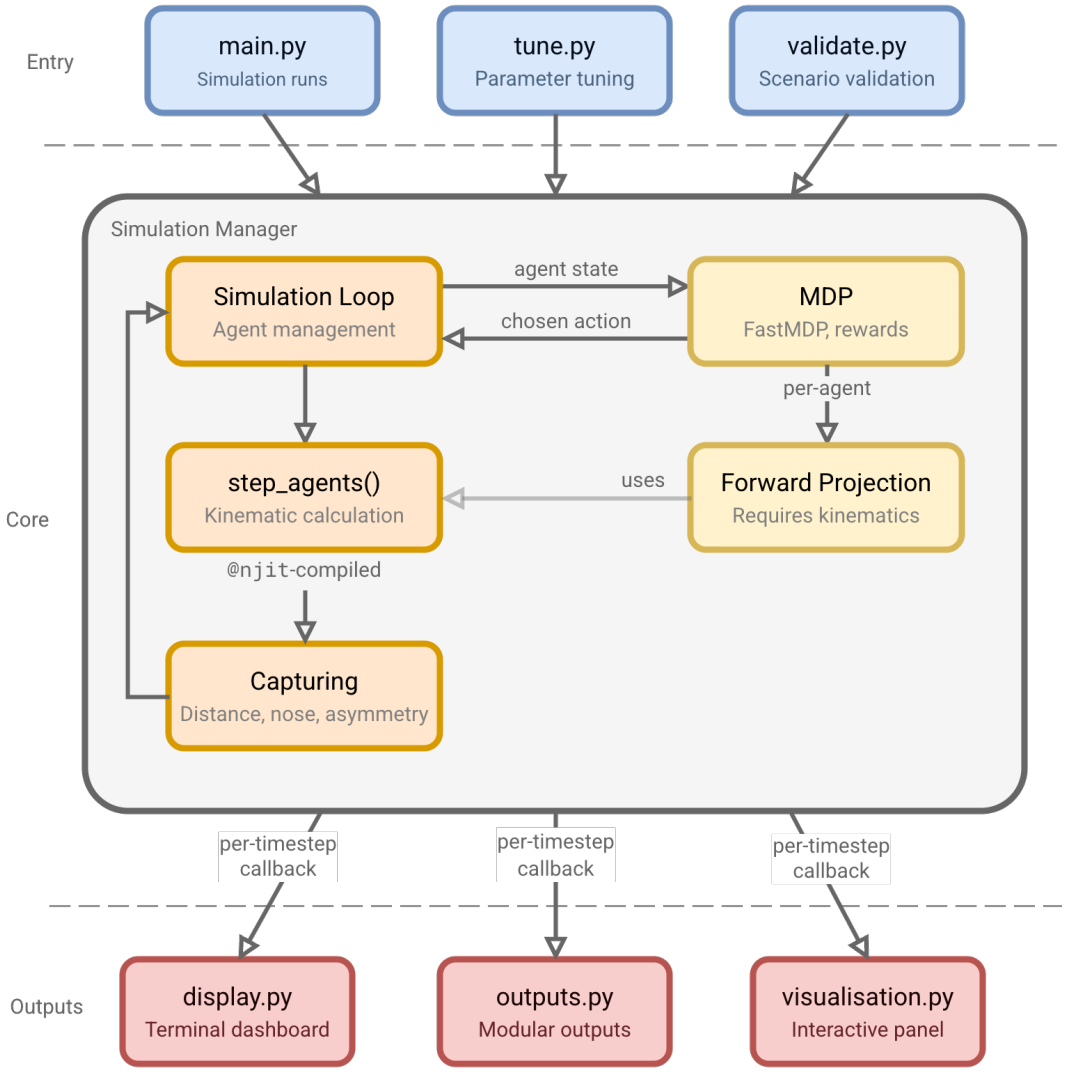

Figure 3.3: System architecture

3.5 Extensions

Milestones Development Timeline In my project proposal I laid out a number of development milestones that would help guide my progress and allow me to assess or adjust my implementations as I continued through the project. I used these as general guidelines for development, but here I am highlighting the actual major milestones that I hit upon throughout the implementation stage.

¡how to include GIT info in this¿?

Milestone 1, 6th November 2025: Repository setup, framework scaffolding and unit tests. ¡more detail¿

Milestone 2, 3rd December 2025: Outputs visualisation setup. Key to examining agent kinematic and decision making behaviour for next steps of development. ¡more detail¿

Milestone 3, 22nd December 2025: First implementation of the FastMDP algorithm and naive reward function setup to drive pursuit-evasion behaviour, initially in one-dimensional space.

Milestone 4, 29th December 2025: 1-dimensional agent competition implementation. Milestone 5, 12th January 2026: 2-dimensional agent competition implementation. Milestone 6, 22nd January 2026: 3-dimensional agent competition implementation including 6DoF kinematic behaviour. These three milestones demonstrate the progression from a one-dimensional space to three-dimensional space. As described in Preparation this is a deviation from my proposal timeline, as completing the base implementation of FastMDP allowed me to progress incrementally and ensure that the pursuit-evasion behaviour remained valid.

Milestone 7, 12th February 2026: Final reward functions

Milestone 8, 31st March 2026: Finished parameter tuning investigations and behaviour validation implementations, fully completing the project core development. These were key for looking ahead to my evaluation stage, where I would need to have data on how I explored different dominant parameters and how they affected my pursuit-evasion decisions and agent behaviour validity.

¡extension milestones here, e.g¿ ¡Milestone 9, completed core evaluation/diss draft so explored extensions A, B, C¿

Final Project Exploration After my project core was completed I generated a system diagram to demonstrate the components of the project.

¡diagram¿

‘main.py‘ This is the primary entry point to the program, and is used for individual configurable simulation runs. It initialises logging and sets up a command-line interface to manage visualisations, outputs and displays.

‘simulation.py‘ This contains the simulation manager - a wrapper around simulation runs to manage step callbacks and parameter tuning - as well as the core simulation framework.

‘mdp.py‘ This contains my implementation of the FastMDP algorithm from [1]. The logic matches to the pseudo-code provided in the paper but the implementation is my own and

leverages numpy arrays to. I took a totally separate approach to developing and refining my reward functions, described further in the Parameter Tuning section.

The ‘argparse‘ standard library allowed me to set up an intuitive terminal interface to manage the additional aspects of the project outside of the simulation framework. Including this infrastructure was a key part of my software development methodology as it allowed me to explore and debug the computational aspects more comprehensively throughout development, described further in the Development Strategy section.

The simulation manager is in charge of managing an individual simulation run. A run continues until it reaches a configurable maximum number of timesteps or only one agent remains uncaptured. The manager also accepts a set of callback functions that get called after each timestep, and are able to access the state of the simulation at that point.

The final version of the positive and negative rewards are included here. We can see that each agent has access to the following parameters: Its own position Its enemy's position Its enemy's velocity

Parameter Tuning As mentioned in Implementation I performed analysis of my model parameters to develop the reward functions. What parameters did I investigate What effect did they have Did I find any dominant parameters - show this Graphs tables of parameter trade-offs.

(quickly): Multiple rounds of behaviour appraisal. I tweaked the reward functions especially, talk about journey from just position -¿ positions velocity -¿ headings etc all influenced by the parameter tuning and tracking. What did I see that led to my clear progression towards correctness.

Development Strategy

Agile Development. I adapted an agile methodology to suit my needs as a solo software developer.

Test Driven Development. I decided I wanted to leverage generative AI tooling to help with developing my project, and decided to l

Extensions Possibly explore other decision frameworks, e.g search and rescue or collaborative flight Could get a lot of marks for this! Breadth of the applicable of field.

Contributions to the Field The project has applications to Collaborative Strategies: A key extension of the project was to implement collaboration between teammate agents. Novel reward functions extended baseline research to Automated Airflight: The project contains a comprehensive, vectorised simulation of 3-dimensional, pseudo-6DoF airflight, implemented in Python and JIT compiled with numba for further optimisation. This codebase can be separated from the decision making process and utilised in other environments and problem spaces. This is expanded on in ¡Future Work¿. JIT-Accelerated Python: Empirical (plots, figure, functional performance etc.) analysis of application to continuous state space problems.

Chapter 4

Evaluation

Chapter 5

Conclusions

Bibliography

[1] Jason Bertram and Jianghai Wei. "Scalable Online Planning for Multi-Agent MDPs". In: *arXiv preprint* (2019). arXiv: 1909.04171.

Appendix A

⟨Sprint Reviews⟩

Sprint reviews!

Appendix B

⟨Configuration Values⟩

Appendix B...

Appendix C

⟨Validation Scenarios⟩

— Scenario — Initial Separation — Relative Heading — Velocity Ratio (P:E) — Altitude — ——————————— — 1 — Far (4000m) — Head-on — Equal (0.2 vs 0.2 Mach) — Mid (5000m) — — 2 — Close (500m) — Head-on — Equal (0.2 vs 0.2 Mach) — Mid (5000m) — — 3 — Mid (2000m) — Facing away — Equal (0.2 vs 0.2 Mach) — Mid (5000m) — — 4 — Mid (2000m) — Perpendicular — Equal (0.2 vs 0.2 Mach) — Mid (5000m) — — 5 — Mid (2000m) — Head-on — Pursuer faster (0.3 vs 0.15 Mach) — Mid (5000m) — — 6 — Mid (2000m) — Head-on — Pursuer slower (0.15 vs 0.3 Mach) — Mid (5000m) — — 7 — Mid (2000m) — Head-on — Equal (0.2 vs 0.2 Mach) — Low (15m) —

Appendix D

Proposal

Proposal...